

Bloom Classifier (sklearn track)

Session Findings

Offline Python trainer: data pipeline, evaluation design, and experiments

Period: 2026-09-19 to 2026-09-27 | Author: sklearn training track

Code: scripts/bloom/ | Artifacts: data/bloom/

1. Tokenizer (PHP-parity, frozen in the JSON contract)

Both trainers implement the identical pipeline (BloomClassifierService.php:54-101, mirrored in scripts/bloom/train_bloom_sklearn.py). Step by step on a raw objective string:

- 1. Lowercase the whole string.
- 2. Replace every non a-z character with a space (re [^a-z\s] -> ' '). Digits, punctuation and apostrophes are removed; e.g. Tagalog contractions split (iba't -> 'iba t').
- 3. Split on whitespace (after trim).
- 4. Drop tokens of length <= 2. Removes is/to/of but also Tagalog function words (ng, sa, ay, na).
- 5. Append bigrams joined with UNDERSCORE (token[i]_token[i+1]), e.g. identify_root, root_cause. A custom CountVectorizer analyzer is used, NOT ngram_range (which joins with space and would break vocab lookup against PHP).
- 6. Vectorize: each gram looks up a frozen sorted vocab (min_df filtered) and increments a raw count (vec[idx] += 1.0). No TF-IDF, no binary features.

Vectorization happens twice with data/bloom JSON as the frozen bridge: at TRAIN time (CountVectorizer.fit_transform builds the vocab: 7,184 entries at min_df=5) and at INFERENCE time per classify() call (featurize + dot product + softmax in PHP). Any word unseen in training is silently ignored at inference, which is why vocab coverage (anchor + PH rows) matters more than the algorithm. L2-regularized multinomial logistic regression (solver=lbfgs, C=0.5) then learns 6 x V weights; the bias/intercept is not penalized and class_weight=balanced rescales the loss per class.

2. Datasets: combined file plus per-source splits

data/bloom/samples.csv holds 9,349 rows (47 exact-duplicates dropped, 0 overlaps with the 60-sentence holdout v2). build_samples_csv.py now also writes one file per source (same text,label,source columns); the combined file is unchanged:

Source	Rows	Content	Train weight
kaggle (vijaydevane, 8,767 Qs)	8,720	Exam questions BT1-BT6 rewritten to objectives via template	1x
ph_k12 (jsano, 35 topics x 6)	210	Philippine Science/History/English questions	2x
anchor (BloomTrainingData.php)	419	ICT/Tagalog/DepEd objectives	8x (live model)

Effective fit rows for the live model: 12,492. Level mix of the base file is imbalanced (Remembering 2,671 / Understanding 1,895 / Applying 1,607 / Analyzing 1,392 / Evaluating 884 / Creating 900); the skew comes entirely from kaggle (BT1 2,582 vs BT5 783) while anchor (~65-88/level) and ph_k12 (exactly 35/level) are balanced. Deliberate exclusions: EDM 21k (no public release found), frankwong2001 (only synthetic ssf-* sets, no blooms set), CogBench silver 26k (user decision; gold 739 kept eval-only), itsskofficial/falcon-7b 3,902 (label collapse: Evaluation 19, Synthesis 2).

Vocabulary synergy

Combined vocab (7,184) exceeds the sum of per-source vocabs (4,793 + 205 + 95 = 5,093). The ~2,000 surplus grams sit below min_df inside every single source but pass it jointly (e.g. 2x in kaggle + 3x in anchor). The mixture retains shared ICT/DepEd phrasing that each source alone would discard as noise.

3. The 60/40 stress split

Every training run now reports three stratified splits (seed 42) plus 5-fold CV: 80/20 and 70/30 per the plan, plus a 60/40 split that trains on only 60% and tests on 40% (4,997 rows). It is designed to punish memorization: small-data artifacts look heroic at 80/20 and collapse at 60/40. Results for the live configuration (counts, C=0.5, anchor x8):

Split	Accuracy	Macro F1	Train-val gap
80/20	0.841	0.856	0.10
70/30	0.838	0.854	0.10
60/40	0.828	0.844	0.12
5-fold CV macro F1	-	0.855 +/- 0.003	-

Degradation from 80/20 to 60/40 is only 0.012 macro with the overfit guard (train-val > 0.15) quiet throughout: the combined model is genuinely stable, not memorized.

4. TF-IDF experiment (counts win, stays diagnostic)

Motivation: the Kaggle rewrite template repeats 'Students will be able to' across 8,767 rows, so IDF-style down-weighting of boilerplate looked promising. Method: TfidfVectorizer with the identical custom analyzer, same C/min_df/seed/upweights (--vectorizer tfidf). Outcome:

Split / metric	Raw counts	TF-IDF
80/20 acc / macro	0.841 / 0.856	0.830 / 0.840
70/30 acc / macro	0.838 / 0.854	0.820 / 0.830
60/40 acc / macro	0.828 / 0.844	0.815 / 0.830
5-fold CV macro	0.855 +/- 0.003	0.837 +/- 0.003
Train-val gap	~0.10-0.12	~0.04-0.05

TF-IDF loses 1-2 points on every accuracy metric; its only win is a smaller train-val gap (row normalization constrains weights, but that calibration does not convert to better predictions). Interpretation: L2 (C=0.5) already shrinks boilerplate weights toward zero, so IDF re-solves a solved problem while slightly blurring real cue strengths. Verdict: live model stays on raw counts. Structural guard: the trainer REFUSES tfidf export to any storage/ path (PHP inference is raw-count only), so a diagnostic vectorizer can never accidentally become the live model. The TF-IDF candidate is retained under data/bloom/ for the appendix only.

5. Per-source runs (same C=0.5, counts, weights 1x, --no-export)

Source (rows/vocab)	80/20 macro	60/40 macro	CV macro	Guard
kaggle (8,720 / 4,793)	0.79	0.78	0.794 +/- 0.004	TRIPS all splits
anchor (419 / 205)	0.83	0.84	0.861 +/- 0.021	quiet
ph_k12 (210 / 95)	0.91	0.81	0.898 +/- 0.036	TRIPS at 60/40
combined (live)	0.856	0.844	0.855 +/- 0.003	quiet

- kaggle = scale but noisy: the rewrite template ('Students will be able to about what proportion...?') leaves grammatical artifacts; the guard trips on every split. Alone it underperforms the mixture (0.79 vs 0.84+).
- anchor = quality: only 419 rows yet the most stable source (60/40 IMPROVES on 80/20). This explains the old PHP baseline's 98.3% in-distribution holdout score.
- ph_k12 = mirage at small N: 210 near-twin rows (same 35 topics x 6 levels sharing content words) with a 42-row test set flatter 80/20 (0.91); the 60/40 stress exposes it (0.81, gap 0.187, folds swinging 0.86-0.95). Kept for Tagalog/DepEd coverage, never standalone.

- Analyzing is weakest in EVERY source (kaggle 0.68-0.72, anchor 0.80-0.85, combined ~0.78-0.80): a genuine cue-overlap problem (identify/compare/evaluate + technical objects), not a data artifact. The 5 holdout misses need fresh anchor-style Analyzing labels, not reweighting.

6. Deployment state and reproduction

- LIVE model: counts, C=0.5, anchor x8 (storage/app/private/ml/bloom_model.json; gitignored). Holdout v2: 91.7% (55/60). CogBench-gold OOD slice (739 Qs): 81.6% vs old PHP baseline 51.8%.
- Old PHP baseline (419 rows: holdout 98.3%, OOD 51.8%) kept as bloom_model.json.bak.php-baseline-20260919. Rollback = one file copy, no deploy.
- Verification at cutover: Bloom suite 16/16 green (full suite 22/23, one pre-existing ExampleTest 404 also present before this work); 1-per-level PHP spot-check 6/6.
- Reproduce combined run: `python scripts/bloom/train_bloom_sklearn.py --input data/bloom/samples.csv --C 0.5 --anchor-weight 8 [--vectorizer tfidf] [--no-export]`.
- Reproduce one source: same command with `--input data/bloom/samples_<kaggle|ph_k12|anchor>.csv --ph-weight 1 --anchor-weight 1 --no-export --tag <source>`.
- Thesis-ready artifacts per run: `classification_report_<split>[_tag].{csv,tex}`, `confusion_matrix_<split>[_tag].{csv,png}`, `metrics_<timestamp>[_tag].json`.